Fleet Collection - One-Page App Summary

What it is
Fleet Collection is a browser-based Phaser game built with Vite. It is a Kantai Collection-style fleet management and gacha app with switchable Pokemon or anime artwork modes.

Who it's for
Primary persona: players who want a lightweight, session-based collection RPG with gacha pulls, team building, and tactical map battles.

What it does
- Runs a construction gacha with rarity rates, pity counter, and guaranteed higher-rarity behavior in multi-pulls.
- Runs a premium gacha with grand prizes, consolation rewards, and token-based exchange mechanics.
- Lets players organize a 6-slot fleet, assign ships, and manage level/stat progression.
- Supports sortie gameplay: map selection, node navigation, formation choice, and battle phases (including air and night phases).
- Tracks ship HP damage states, repairs, and dock capacity upgrades.
- Maintains a collection view with ownership/completion progress and ship detail panels.
- Includes secret-code redemption for rewards and artwork-mode switching.

How it works (repo-evidenced architecture)
- Client app: Vite-served Phaser 3 single-page app (`src/main.js`, `index.html`).
- Scene layer: UI/gameplay flows in scene modules (boot, title, gacha, premium gacha, battle, map, fleet, collection, dock, secret code, gift/exchange scenes).
- System layer: reusable logic in `src/systems/*` for persistence (`storage.js`), standard gacha (`gacha.js`), premium gacha (`premiumGacha.js`), audio (`audio.js`), and shared UI helpers (`ui.js`).
- Data layer: static game content in `src/data/*` (ships, enemies, maps, formations, equipment, theme, mapping).
- Assets: images/audio loaded from `public/assets/*` by `BootScene`, with folder selection based on stored artwork mode.
- Persistence/data flow: player input in scenes -> system functions mutate localStorage save data -> scenes read updated state and render next view.
- Backend/API services: Not found in repo.

How to run (minimal)
1. Install dependencies: `yarn install`
2. Start dev server: `yarn dev`
3. Open the app at `http://localhost:3000` (Vite is configured for port 3000; auto-open is enabled).

Notes
- README/getting-started documentation file: Not found in repo.